Software Design Specification

The Last Stronghold

Prepared in conformance with the organizational requirements of IEEE Std 1016-2009 using only the project evidence contained in the uploaded IEEE standard and the "The Last Stronghold Blueprint Breakdown Report" source document.

Prepared for: Software Engineering Capstone Review
Project: The Last Stronghold
Platform: Unreal Engine 5 (Blueprint-only project)
Authors: Christian Alvarez, Stanley Delva, Hendrick Christian
Document status: Draft for academic submission
Version: 1.0
Date of issue: March, 2026

Contents

Frontspiece
Introduction
Purpose
Scope
Context
Summary
References
Glossary
Identified stakeholders and design concerns
Design Viewpoint 1 — Context viewpoint / Design View 1
Design Viewpoint 2 — Composition viewpoint / Design View 2
Design Viewpoint 3 — Dependency viewpoint / Design View 3
Design Viewpoint 4 — Interface viewpoint / Design View 4
Design Viewpoint 5 — Interaction viewpoint / Design View 5
Design Viewpoint 6 — Algorithm viewpoint / Design View 6
Design Viewpoint 7 — Resource viewpoint / Design View 7
Design rationale
Annex A — Source bibliography
Annex B — Design language descriptions
Annex C — Figures

Frontspiece

Date of issue and status: 2026; Draft for academic submission.

Issuing organization: Software Engineering Capstone project documentation package.

Authorship: Christian Alvarez, Stanley Delva, Hendrick Christian

Change history

Version	Date	Description
1.0	2026	Initial SDS

Introduction

Purpose

This document provides a software design specification for The Last Stronghold using the information-content and organization principles described by IEEE Std 1016-2009. The goal is to record, organize, and communicate confirmed design information about the project's Blueprint systems, interfaces, and user-interface behavior without introducing unsupported implementation details.

Scope

The design subject is The Last Stronghold, an Unreal Engine 5 Blueprint-only capstone project. Based on the uploaded blueprint breakdown report, the documented design scope includes the GameMode-driven wave loop, the player character as the central receiver of UI and resource updates, the projectile and damage pipeline, the enemy death-state interface, the defendable tower objective, autonomous turret behavior, explosive hazard behavior, soul-shard pickups, and the four documented UI widgets.

Context

The blueprint breakdown report identifies a complete gameplay loop already present in the project: the GameMode controls waves and win/loss flow; the player and pickup systems manage currency and UI; towers and turrets defend the objective; and explosive support actors apply area damage and cleanup logic. This SDS therefore documents a running gameplay architecture rather than a hypothetical future design.

Summary

In IEEE 1016 terms, this SDS identifies stakeholders and design concerns, selects the viewpoints that are demonstrably applicable to the available source evidence, and presents design views for each selected viewpoint. The document includes four requested visual artifacts: a UML class diagram, a system architecture diagram, a sequence diagram, and a state machine. Each figure is explicitly grounded in the blueprint breakdown report and is framed as a documentation abstraction of confirmed behavior.

References

- IEEE Std 1016-2009, IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.
- The Last Stronghold Blueprint Breakdown Report, uploaded project source document.

Glossary

Term	Meaning in this SDS

Blueprint	Unreal Engine visual scripting asset used to implement documented project behavior.
Design concern	An area of interest in the software design, as defined by IEEE 1016.
Design view	A representation addressing a set of design concerns from a selected viewpoint.
Design viewpoint	The specification of elements and conventions used to build a design view.
HUD	The player-facing heads-up display documented through W_MainHUD.
Interface call	Blueprint-interface communication used to avoid hard references where confirmed by the report.

Identified stakeholders and design concerns

IEEE 1016 requires identification of design stakeholders and their concerns. Only concerns that are evidenced by the uploaded materials are listed below.

Stakeholder-to-concern matrix

Stakeholder	Confirmed design concerns
Developer / maintainer	Need modular systems, clear responsibilities, and a single source of truth for wave progression.
Player	Needs visible feedback for enemies remaining, wave number, soul shards, round countdown, health, and win/lose outcome.
Tester / reviewer	Needs traceable event flow for damage, win/loss, pickups, turret targeting, and UI updates.
Project instructor	Needs a design description that is organized, explicit, and grounded in documented implementation evidence.

Across the source material, the most persistent design concerns are the following:

- Correct and centralized wave progression.
- Safe communication between systems through interfaces rather than unnecessary hard references.
- Reliable enemy-death handling so turrets do not continue attacking invalid or already-dead targets.
- Clear player-facing feedback through the HUD and end-of-level widgets.
- Contained area-damage logic with explicit cleanup after impact or detonation.
- Single-use protection for pickups and explosives through DoOnce-style gating.

Design Viewpoint 1 — Context viewpoint

Viewpoint specification

Field	Content
Viewpoint name	Context viewpoint
Design concerns	System boundary, primary runtime actors, high-level gameplay services, player-facing flow.
Design elements used	Player, GameMode, enemies, defense actors, pickup actor, UI widgets, and confirmed information flows among them.

Analytical use	Clarifies what the design subject does at a black-box / subsystem level before component-level details are discussed.
Viewpoint source	IEEE Std 1016-2009, Clause 5.2.

Rationale for selection

The blueprint breakdown report explicitly describes the project as a coordinated gameplay loop involving GameMode control, enemy waves, player interaction, currency collection, objective defense, and UI updates. Those features make the Context viewpoint applicable because it shows the system's externally meaningful services and the principal actors that participate in them.

Design View 1 — Context view of the implemented gameplay loop

Figure 1 documents the project as a set of interacting runtime subsystems. The diagram is not a guessed engine architecture; it is a documentation abstraction of the exact roles stated in the breakdown report: GM_TowerDefense as wave authority; GideonPlayerCharacter as the main playable actor and central receiver of interface-driven updates; BP_Skeleton as the documented enemy actor; defense actors comprising BP_Turret, BP_Turret_Blast, BP_Explosive, and BP_Tower; BP_SoulShardPickup as the resource pickup; and the documented widgets as the presentation layer.

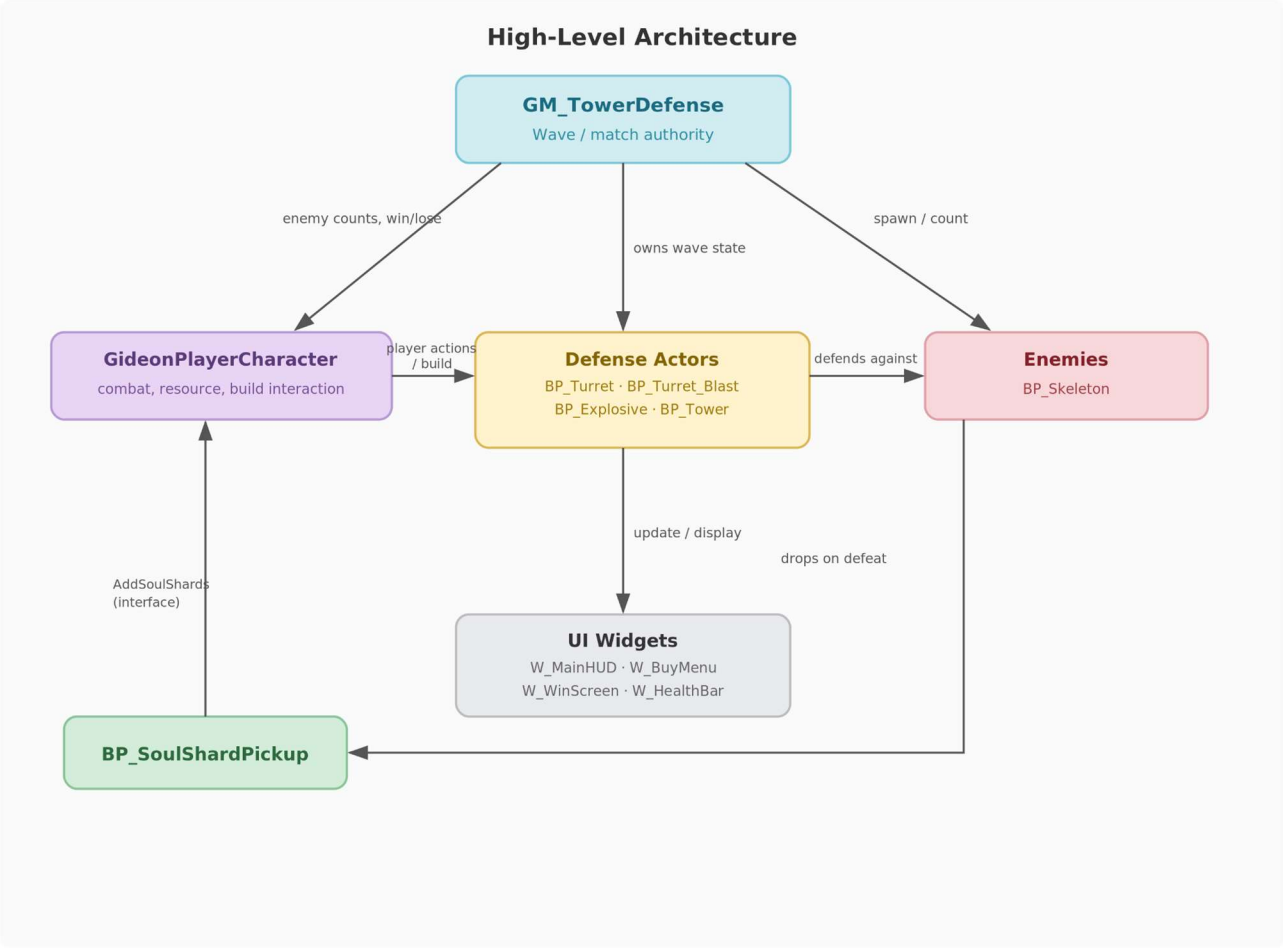

Figure 1 — System architecture diagram derived from the confirmed subsystem roles in the blueprint breakdown report.-

Design Viewpoint 2 — Composition viewpoint

Viewpoint specification

Field	Content
Viewpoint name	Composition viewpoint
Design concerns	How the design subject is organized into major parts and what each part is responsible for.
Design elements used	Subsystems, Blueprint actors, widgets, and the part-whole organization of the gameplay loop.
Analytical use	Helps reviewers understand where behavior lives and prevents the design from being read as one undifferentiated set of Blueprints.
Viewpoint source	IEEE Std 1016-2009, Clause 5.3.

Rationale for selection

The source report is heavily component-oriented: each documented Blueprint is described with a purpose, primary flow, and design notes. The Composition viewpoint is therefore one of the strongest fits for the available evidence.

Design View 2 — Compositional structure

Compositional decomposition of the design subject

Area	Elements	Confirmed role
Game management	GM_TowerDefense	Owns spawn-point references, tracks active and remaining enemies, advances waves, and triggers win/loss flow.
Player	GideonPlayerCharacter	Primary player-controlled character responsible for movement, combat-related interaction, tower-defense interaction, and receiving UI/resource updates through interfaces.
Projectile / enemy	BP_Projectile; BP_Skeleton	Projectile transfers ranged damage; skeleton exposes death state through GetIsEnemyDead?.
Defense	BP_Tower; BP_Turret; BP_Turret_Blast; BP_Explosive	Objective health and loss trigger; autonomous targeting and firing; compact blast damage; larger detonating hazard damage.
Resources	BP_SoulShardPickup	Collectible currency pickup that grants soul shards and destroys itself.
UI	W_MainHUD; W_BuyMenu; W_WinScreen; W_HealthBar	Presentation layer for combat info, build selection, end-of-level flow, and reusable health display.

Composition notes

- GM_TowerDefense is documented as the backbone of the tower-defense loop and the single source of truth for wave progression.
- GideonPlayerCharacter is documented as the center of HUD, resource collection, and tower-defense interaction.
- Defense behavior is deliberately split between sensing/tracking (BP_Turret), impact payload (BP_Turret_Blast), and objective state (BP_Tower).
- Widgets are presentation objects rather than match-authority objects; the report explicitly states that W_MainHUD receives event-driven values rather than owning match logic.

UML Class Diagram

IEEE 1016 does not require UML specifically, but it allows standardized design languages to be selected when they support the chosen viewpoint. Because the uploaded IEEE standard lists UML among accepted languages and because the user requested a UML class diagram, Figure 2 models only those attributes, functions, and dependencies that are explicitly confirmed in the blueprint breakdown report.

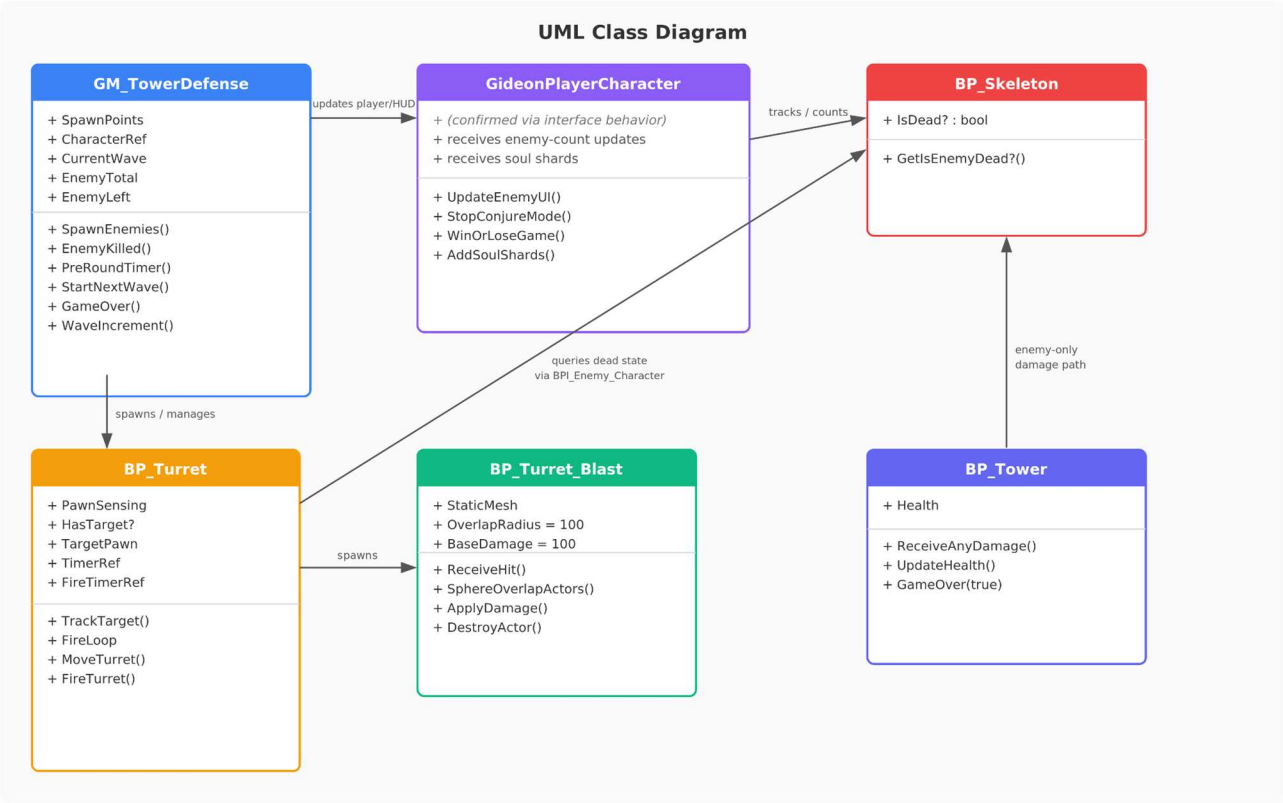

Figure 2 — UML class diagram using only confirmed class members, functions, and inter-Blueprint dependencies named in the source report.

Design Viewpoint 3 — Dependency viewpoint

Viewpoint specification

Field	Content
Viewpoint name	Dependency viewpoint
Design concerns	Interconnection, control flow, and safe communication among entities.
Design elements used	Uses/provides/requires relationships evidenced by spawning, interface calls, timer callbacks, and UI updates.
Analytical use	Shows how state and events move through the design, which is critical for debugging and testing.
Viewpoint source	IEEE Std 1016-2009, Clause 5.5.

Design View 3 — Confirmed dependencies

Dependency inventory

Dependency	Grounded evidence
GM_TowerDefense → player/HUD	Enemy deaths update counters and UI; GameMode communicates with the player HUD through the main-character interface.
BP_Tower → GameMode	On tower destruction, GameOver(true) is called through the tower-defense GameMode interface.
BP_Turret → BP_Skeleton	FireTurret validates TargetPawn and queries GetIsEnemyDead? through BPI_Enemy_Character before spawning an attack.
BP_Turret → BP_Turret_Blast	FireTurret spawns BP_Turret_Blast from the Arrow component transform.
BP_SoulShardPickup → GideonPlayerCharacter	Pickup calls AddSoulShards through BPI_MainCharacter on the overlapping player actor.
W_BuyMenu → GideonPlayerCharacter	Menu button presses call PlaceBuildable through BPI_MainCharacter with buildable index and cost.
Actors with widgets → W_HealthBar	Health-owning actors update or hide the widget through BPI_HealthBarInterface.

The dependency structure shows two consistent patterns. First, the design avoids unnecessary direct ownership by using interfaces for cross-system updates. Second, the GameMode and objective tower form the authoritative gameplay-end path, while the turret and enemy getter form a defensive safety path that prevents continued attacks on invalid or already-dead targets.

Design Viewpoint 4 — Interface viewpoint

Viewpoint specification

Field	Content
Viewpoint name	Interface viewpoint
Design concerns	How cooperating entities communicate and what a receiving entity must expose.
Design elements used	Documented interface calls and widget event entry points.
Analytical use	Supports integration, testing, and maintainability by clarifying contract boundaries.
Viewpoint source	IEEE Std 1016-2009, Clause 5.8.

Design View 4 — Interface contracts in the confirmed design

Interface inventory

Interface / entry point	Confirmed responsibility
BPI_Enemy_Character.GetIsEnemyDead?	BP_Skeleton exposes current IsDead? boolean to outside actors, especially turrets.
BPI_MainCharacter.AddSoulShards	BP_SoulShardPickup grants currency to the overlapping player actor through the interface.
BPI_MainCharacter.PlaceBuildable	W_BuyMenu forwards build selection to the player with BuildableIndex and Cost.
BPI_HealthBarInterface.UpdateHealth	Actors such as enemies or the tower update the progress bar using a normalized percentage path.
BPI_HealthBarInterface.HideHealthBar	Owning actor hides the widget when appropriate.
Player-facing widget entry points	W_MainHUD receives UpdateAmmoText, UpdateEnemiesLeft, UpdateWaveText, UpdateSoulShardsText, StartNextRoundEvent, and SetWinOrLose.

The strongest design characteristic visible here is decoupling. The report repeatedly notes that systems treat the player through interfaces instead of hard references wherever possible. That choice is consistent with the maintainability concerns identified earlier and is directly supported by the pickup, buy-menu, and health-bar implementations.

Design Viewpoint 5 — Interaction viewpoint

Viewpoint specification

Field	Content
Viewpoint name	Interaction viewpoint
Design concerns	Event ordering, messaging, and runtime collaboration among actors.
Design elements used	Unreal events, timer callbacks, interface queries, and spawned attack actors.
Analytical use	Makes multi-step Blueprint flows understandable to reviewers and testers.
Viewpoint source	IEEE Std 1016-2009, Clause 5.10.

Design View 5 — Turret targeting and blast sequence

The most fully documented multi-step interaction in the source report is the BP_Turret targeting and firing flow. It begins with PawnSensing.OnSeePawn, continues through target validation and timer setup, uses MoveTurret for repeated aiming, checks GetIsEnemyDead? before firing, and then spawns BP_Turret_Blast, which resolves area damage and destroys itself.

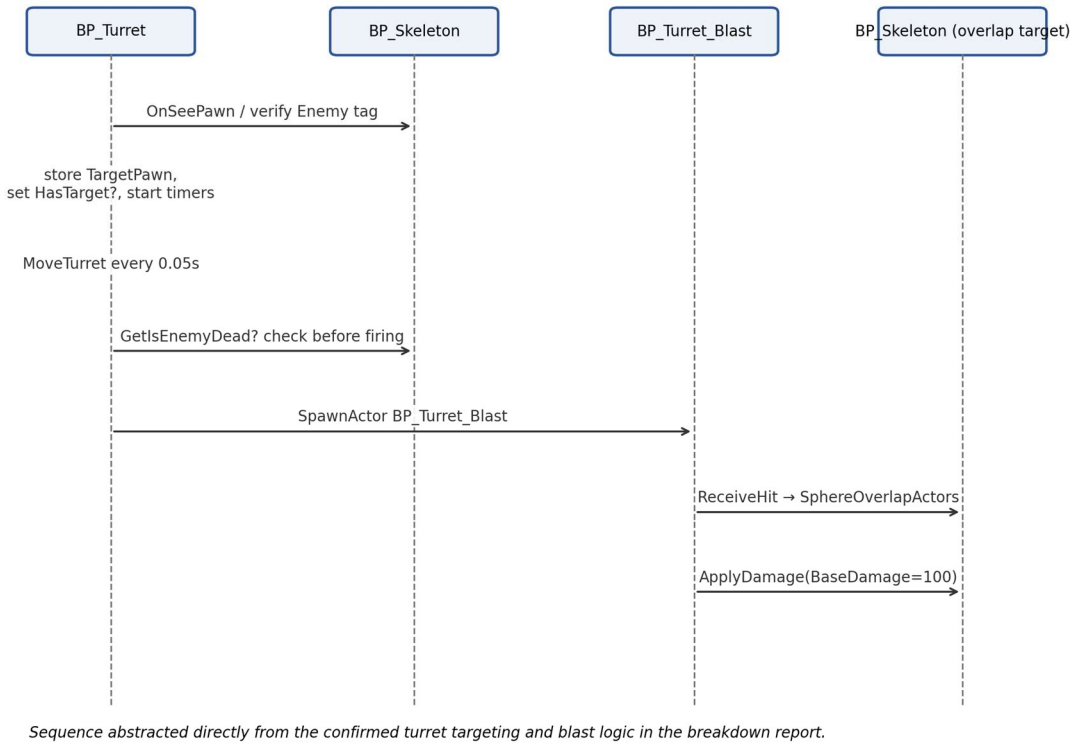

Figure 3 — Sequence diagram for the confirmed turret-targeting and blast-resolution flow.

A second interaction path, also confirmed in the report, is the enemy-death / UI update path: enemy deaths update counters in GM_TowerDefense, and the GameMode can push those values back to the player interface so the HUD stays synchronized. Because the report does not enumerate every intermediate node of that path, this document describes it textually rather than inventing a more detailed sequence diagram than the evidence supports.

Design Viewpoint 6 — Algorithm viewpoint

Viewpoint specification

Field	Content
Viewpoint name	Algorithm viewpoint
Design concerns	Procedural logic and decision rules inside key entities.
Design elements used	Conditional branches, timer logic, overlap loops, tag checks, DoOnce gating, and destruction/cleanup logic.
Analytical use	Captures how important decisions are actually made at runtime.
Viewpoint source	IEEE Std 1016-2009, Clause 5.12.

Design View 6 — Key confirmed algorithms

Algorithm catalog

Algorithm	Confirmed processing logic
Wave progression algorithm	BeginPlay gathers spawn points; enemies are spawned; enemy deaths update counters and UI; when a wave ends the round timer starts; the next wave begins or the game ends.
Tower damage algorithm	ReceiveAnyDamage → confirm DamageCauser has tag Enemy → compare Health - Damage against zero → update health and widget → if zero or less, play FX, destroy tower, and call GameOver(true).
Turret tracking algorithm	OnSeePawn → verify Enemy tag and no current target → store TargetPawn / HasTarget → start TrackTarget timer every 0.05 s → rotate using FindLookAtRotation and RInterpTo.
Turret firing algorithm	FireLoop timer calls FireTurret every 3 s → validate target and query GetIsEnemyDead? → if alive, spawn BP_Turret_Blast; otherwise clear timer.
Blast damage algorithm	ReceiveHit → spawn impact emitter → hide mesh → SphereOverlapActors(radius 100) → ApplyDamage(BaseDamage 100) to overlapped actors → DestroyActor.
Explosive detonation algorithm	ReceiveAnyDamage → DoOnce → spawn explosion FX → SphereOverlapActors(radius 300) → ApplyDamage(BaseDamage 100) to overlaps → destroy explosive even if none found.
Pickup algorithm	Sphere overlap begins → verify player tag → DoOnce → AddSoulShards interface call → destroy pickup.
HUD countdown algorithm	StartNextRoundEvent sets CurrentSeconds to 30, makes WaveStartingBox visible, starts a looping 1-second timer, updates text each tick, and hides the box when CurrentSeconds reaches 0.

Design Viewpoint 7 — Resource viewpoint

Viewpoint specification

Field	Content
Viewpoint name	Resource viewpoint
Design concerns	Runtime resources and consumables explicitly mentioned in the source report.
Design elements used	Soul Shards as currency, timer handles, overlap radii, and widget percent normalization.
Analytical use	Documents the limited but important resource-related facts available from the current evidence.
Viewpoint source	IEEE Std 1016-2009, Clause 5.13.

Design View 7 — Confirmed resource-related design facts

Resource-related elements

Element	Confirmed resource characteristic
Soul Shards	Documented as collectible currency added to the player through AddSoulShards and displayed in W_MainHUD.

TimerRef / FireTimerRef	Stored handles in BP_Turret used for periodic tracking and firing loops.
Round timer	W_MainHUD stores TimerRef for the round countdown and clears/invalidates it when the timer reaches zero.
Overlap radius 100	Used by BP_Turret_Blast for compact area damage.
Overlap radius 300	Used by BP_Explosive for larger area damage.
Normalized health display	W_HealthBar multiplies incoming health by 0.01 before setting a progress-bar percent; BP_Tower sends widget updates using normalized values as described in the report.

State Machine

The uploaded materials do not provide a full enemy AI statechart, so this SDS avoids inventing one. Instead, Figure 4 presents a state machine for BP_Turret, because the targeting lifecycle is explicitly documented: idle/no target, target acquisition, tracking, firing, and return to idle when the target becomes invalid or dead.

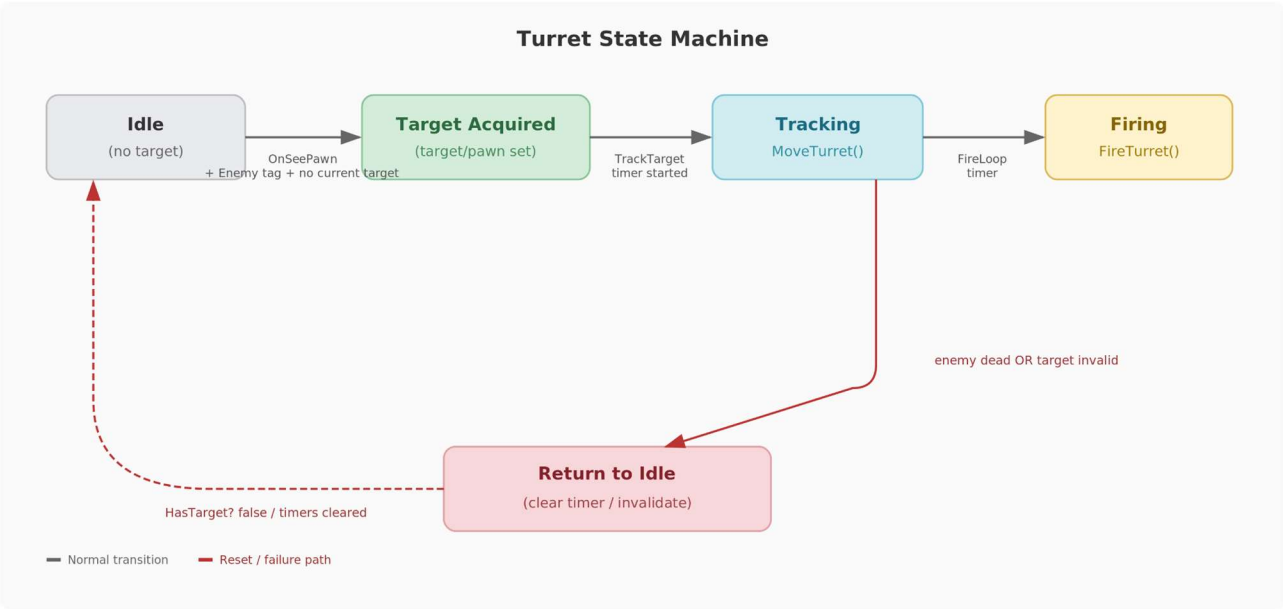

Figure 4 — BP_Turret state machine derived from the documented sensing, tracking, firing, and timer-clearing behavior.

Detailed UI design notes

Because the blueprint breakdown report includes unusually strong detail for the widgets, the user-interface layer warrants its own consolidated documentation section.

Widget catalog

Widget	Confirmed role
W_MainHUD	Displays mana/ammo-style values, enemies remaining, current wave, soul-shard currency, round countdown text, and the win/lose screen state.
W_BuyMenu	Forwards Arcane Turret and Rift Crystal purchase choices to the player character through PlaceBuildable with buildable index and cost.
W_WinScreen	Allows quit game, return to main menu, or restart current level using engine-level functions rather than direct manipulation of gameplay state.

W_HealthBar	Reusable interface-driven progress-bar display that updates or hides based on actor-driven interface calls.

The most important UI design characteristic is separation of presentation from authority. The report explicitly states that W_MainHUD does not own match logic itself; instead, it receives event-driven values from gameplay systems and renders them into player-facing text. That distinction is academically important because it keeps UI logic descriptive rather than authoritative.

Design rationale

IEEE 1016 requires rationale for selected viewpoints and design decisions. The source report provides enough confirmed evidence to support the following rationale statements without speculation:

- Wave progression is centralized in GM_TowerDefense so enemy counts, wave numbers, and win/loss state do not fragment across multiple actors.
- Interface calls are used where cross-system communication is needed, which reduces unnecessary hard coupling between pickups, widgets, enemies, and the player character.
- Turret targeting is split into distinct acquisition, tracking, and firing loops, making the behavior easier to reason about and easier to stop safely when a target becomes invalid.
- Area-damage actors destroy themselves after resolving damage so they remain transient and do not duplicate damage processing.
- Single-use gating through DoOnce protects pickups and explosives from repeated activation in the same overlap or hit window.
- The UI layer is primarily presentational, which keeps player-facing display logic separate from match authority and combat authority.

Just as importantly, this document also records where rationale cannot yet be expanded. The blueprint breakdown report itself states that some sections whose full node exports were not included are summarized from confirmed behavior discussed earlier. In those cases, this SDS deliberately preserves the summary level rather than claiming a more detailed design than the evidence supports.

Annex A (informative) — Source bibliography

- IEEE Std 1016-2009, IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.
- The Last Stronghold Blueprint Breakdown Report (uploaded project document).

Annex B (informative) — Design language descriptions

The IEEE standard permits use of standardized design languages when they are well defined and appropriate to the viewpoint. This SDS uses the following design-language conventions:

Design language inventory

Design language	Used for	Constraint applied in this SDS
Narrative structured prose	Primary design language for all sections	Used because the source evidence is a descriptive blueprint breakdown report.

UML class diagram	Composition / logical clarification	Used conservatively to show only confirmed members and dependencies; no invented inheritance or hidden methods are added.
Sequence diagram	Interaction view	Used only where the event sequence is explicitly documented in the source material.
State machine diagram	Behavioral abstraction	Applied to BP_Turret rather than BP_Skeleton to avoid inventing unsupported enemy states.
Architecture block diagram	Context and composition communication aid	Used to summarize subsystem roles already named in the report.

Annex C (informative) — Figures

Figure register

Figure	Description
Figure 1	System architecture diagram derived from confirmed subsystem roles.
Figure 2	UML class diagram using only confirmed class members, functions, and dependencies.
Figure 3	Sequence diagram for BP_Turret targeting and BP_Turret_Blast resolution.
Figure 4	BP_Turret state machine derived from sensing, tracking, firing, and timer-clearing behavior.